Project - High Level Design

on

Healthcare Planning Assistant Agent

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	MEET SOLANKI	EN22CS301595
02	MAYURI PATIDAR	EN22CS301593
03	MOKSHIKA YADAV	EN22CS301614
04	PALAK SHAHDADPURI	EN22CS301677
05	NIRDESH SHARMA	EN22CS301655
06	PANKAJ	EN22CS301679

Group Name: 11D6

Project Number: DO-22

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Functional Requirements
   7.1. User Authentication & Authorization
   7.2. Infrastructure Configuration Management
   7.3. Provisioning Control & Monitoring
   7.4. Telecom Service Orchestration
   7.5. Dockerfile (Application Containerization)
   7.6. GitHub Actions (Workflow Configuration)
   7.7. CI/CD Strategy (Continuous Integration & Deployment)

8. Non-Functional Requirements

   8.1. Security Aspects

   8.2. Performance Aspects

9. References

10. GitHub

1. Introduction

1.1 Scope of the Document

This document provides a High-Level Design (HLD) for the Healthcare Planning Assistant system. It describes the system architecture, components, workflows, data handling, and integration with external services. The document serves as a blueprint for developers, testers, and stakeholders to understand the system design and its functionalities.

1.2 Intended Audience

- Developers and Software Engineers
- System Architects
- QA/Test Engineers
- DevOps Engineers
- Project Evaluators (Academic/Industry)

1.3 System Overview

The Healthcare Planning Assistant is a FastAPI-based web application that allows users to input symptoms and receive AI-generated healthcare guidance. It uses LLMs (Google Gemini / Groq LLaMA) via LangChain to generate structured medical advice in four sections:

- Possible Conditions
- Precautions
- Medication Guidance
- Nutrition Advice

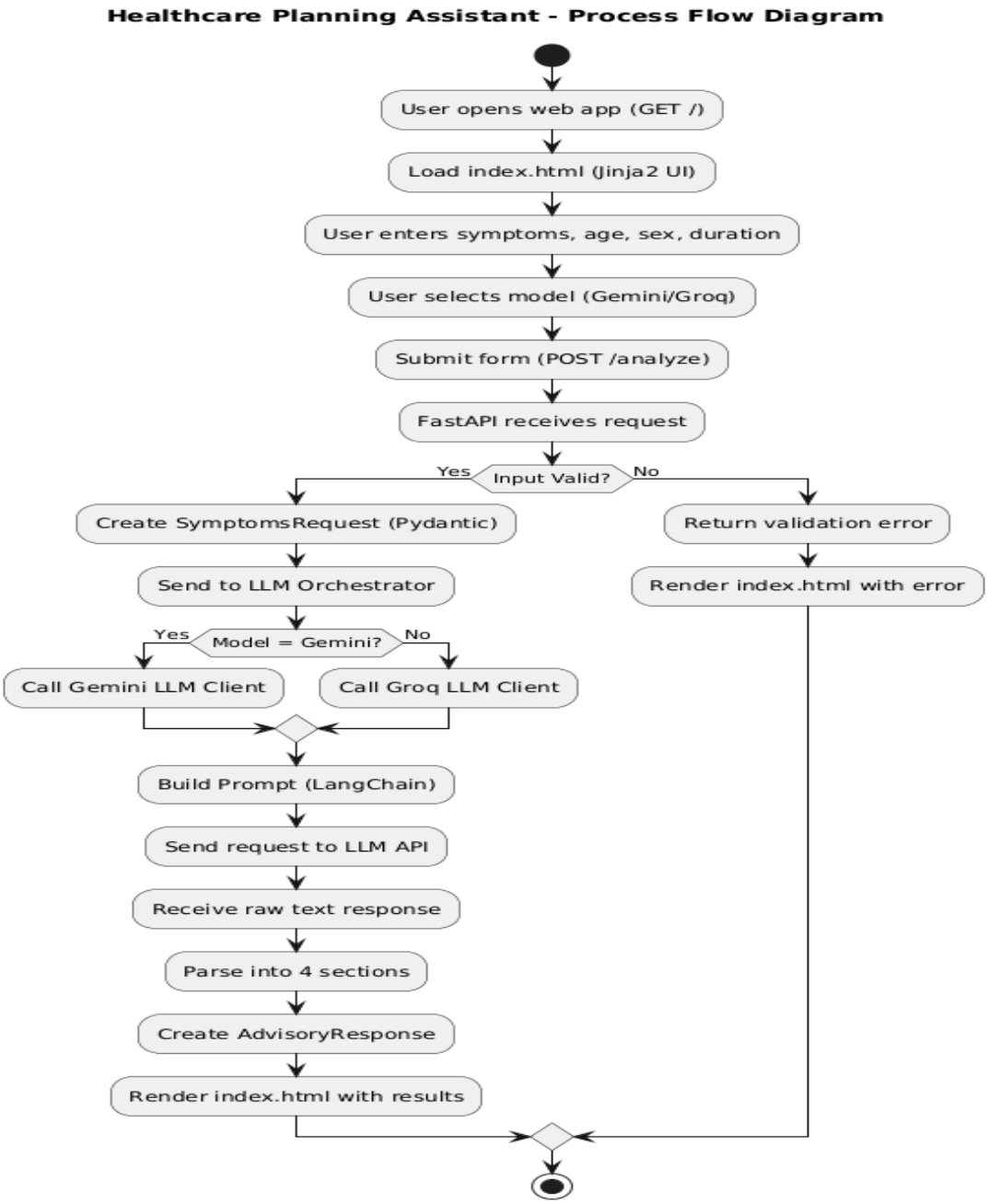
2. System Design

2.1 Application Design

The application follows a **layered architecture**:

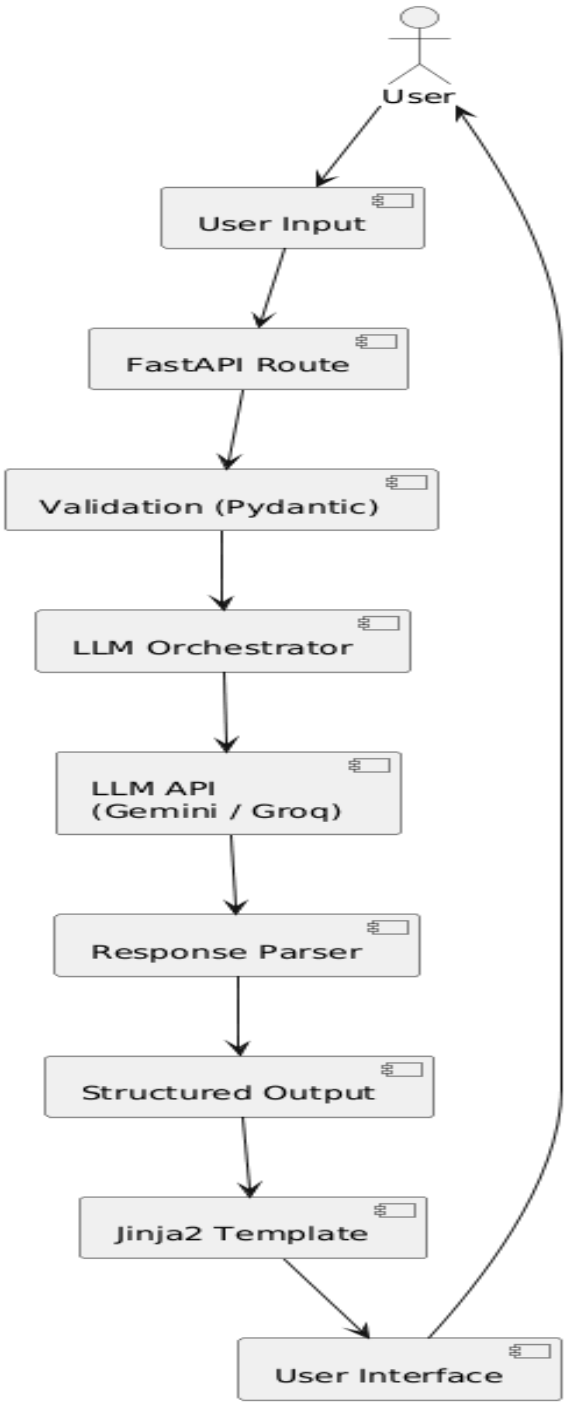
- **Presentation Layer**: Jinja2 templates + Bootstrap UI
- **API Layer**: FastAPI routes
- **Service Layer**: LLM orchestration
- **Domain Layer**: Pydantic models (DTOs)
- **Configuration Layer**: Environment-based settings

2.2 Process Flow



2.3 Information Flow

2.3 Information Flow - Healthcare Planning Assistant



2.4 Components Design

Frontend (UI)

- Jinja2 templates (HTML rendering)
- Bootstrap for styling

Backend

- FastAPI framework
- Routing module (`routes_symptoms.py`)

Service Layer

- LLM Orchestrator
- Gemini Client
- Groq Client

Configuration

- Environment variables (.env)
- Settings loader

External Systems

- Google Gemini API
- Groq LLaMA API

2.5 Key Design Considerations

- **Stateless Architecture** (No database)
- **Modular Design** (Separation of concerns)
- **Provider Flexibility** (Gemini / Groq switch)
- **Error Handling & Validation**
- **Security of API Keys (user-provided at runtime)**
- **Scalability via containerization**

2.6 API Catalogue

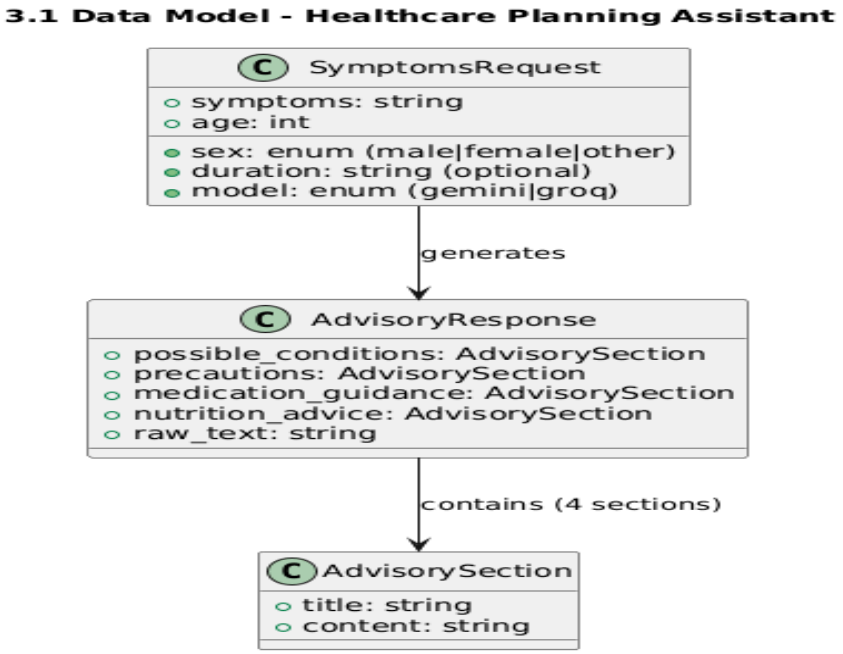
Endpoint	Method	Description
`/`	GET	Load homepage
`/analyze`	POST	Analyze symptoms
`/health`	GET	Health check

3. Data Design

3.1 Data Model

No persistent database is used. Data is handled via in-memory DTOs:

- **SymptomsRequest**
- **AdvisoryResponse**



3.2 Data Access Mechanism

- Data is passed via HTTP requests (form submission)
- Processed in-memory using Pydantic models
- No database queries involved

3.3 Data Retention Policies

- No data is stored permanently
- API keys are entered at runtime and not saved
- Responses exist only during request lifecycle

3.4 Data Migration

- Not applicable (no persistent storage)

4. Interfaces

- **User Interface**: Web-based HTML form
- **API Interface**: REST endpoints (FastAPI)
- **External APIs**:
  - Gemini API
  - Groq API

5. State and Session Management

- The system is **stateless**
- No session storage or cookies used
- Each request is independent

6. Caching

- Currently not implemented
- Future enhancement:
  - Cache repeated queries
  - Use Redis for performance optimization

7. Functional Requirements

7.1 User Authentication & Authorization

- Not implemented in current version
- Future scope: login/signup system

7.2 Infrastructure Configuration Management

- Managed via `.env` file
- Includes API keys and model settings

7.3 Provisioning Control & Monitoring

- Health endpoint `/health`
- Logs for debugging

7.4 Telecom Service Orchestration

- Not applicable to this system

7.5 Dockerfile (Application Containerization)

- Application can be containerized using Docker
- Ensures portability and consistency

7.6 GitHub Actions (Workflow Configuration)

- Automates build, test, and deployment
- Ensures CI pipeline

7.7 CI/CD Strategy

- Continuous Integration: Code validation & testing
- Continuous Deployment: Auto deployment to server/cloud

8. Non-Functional Requirements

8.1 Security Aspects

- API keys are not stored or logged
- Input validation using Pydantic
- CORS configuration
- Protection against invalid inputs

8.2 Performance Aspects

- FastAPI ensures high performance
- Async LLM calls improve efficiency
- Lightweight architecture (no DB overhead)

9. References

- FastAPI Documentation
- LangChain Documentation
- Google Gemini API Docs
- Groq API Docs
- Bootstrap Documentation
- Group Name: **11D6- Healthcare Planning Assistant Agent**

10. Github

- Github link - https://github.com/MeetSolanki0/Dockerized-Healthcare-Service